

Review & Audit — GSE iOS, first pass

Date: 2026-09-15 · **Scope:** everything produced in this session **Method:** adversarial self-review against the repo's own doctrine, then a verification-integrity audit, then upstream findings.

1. What was actually built

Deliverable	Path	State
Expo SDK 57 / RN 0.86 app, iOS-first	<code>app/</code>	41 source files, lint-clean
FIELD design tokens ported natively	<code>app/src/theme/</code>	Complete, incl. dual-mode (Field/Paper)
Trust predicates (adverse edge, staleness, ranking)	<code>app/src/lib/trust.ts</code>	Ported with the upstream asymmetry preserved
Calibration maths (Clopper-Pearson, Brier, discrimination)	<code>app/src/lib/calibration.ts</code>	Independent implementation, reference-checked
HTTP client (retry, gates, labelled cache)	<code>app/src/api/client.ts</code>	27 tests
Runtime response decoders	<code>app/src/api/decoders.ts</code>	Tolerant of new fields, strict on required ones
13 screens incl. age gate, paywall, account deletion	<code>app/app/</code>	All render; cannot be run here
Brand-voice linter bound to the repo vocabulary	<code>app/src/lib/voice.ts</code>	Parity-tested against the server file
Invariant linter + self-test	<code>app/scripts/lint-rules.js</code>	8/8 rules fire on synthetic fixtures
Test suite	<code>app/tests/</code>	105 passing
Server patches (4 routes + bearer auth)	<code>server-patches/</code>	PR-ready
App Store submission packet	<code>docs/store/SUBMISSION.md</code>	Complete
Design review harness	<code>preview/</code>	6 frames, rendered and reviewed

2. Verification — and its limits, stated plainly

This is the most important section of this document.

What IS verified

Check	How	Result
Pure-logic correctness	105 assertions under <code>node --test</code>	Pass
Banned vocabulary	Linters bound to <code>positioning-vocab.json</code>	Pass, and the linter self-tests
Token existence	Linters compares every <code>t.type.X</code> / <code>t.colors.X</code> against the real maps	Pass
Unused imports, <code>any</code> , <code>console.*</code> , <code>require()</code> in ESM	Linters	Pass
Route files export a default	Linters	Pass
Delimiter balance (catches truncated edits)	Linters	Pass
Clopper–Pearson correctness	Published reference values (20/30 $\rightarrow$ [0.4719, 0.8271])	Pass
Entitlement parity with the server	Test reads <code>packages/types/src/ index.ts</code> and compares	Pass
Visual layout at 393pt	Rendered in the harness and looked at	3 defects found and fixed
Core typecheck (13 pure files)	<code>scripts/typecheck.js</code> , which prints its own summary	0 diagnostics (completed run)
Full typecheck	Same	Completed three times with 0 diagnostics, then became unreproducible. Not counted as a gate. See §2.2.

What is NOT verified, and why

1. **The app has never been built or run.** There is no Xcode and no simulator on this host. The Swift/Metro/StoreKit integration, navigation, gestures, haptics, push delivery, and the Live Activity are all **unexecuted code**. They are written carefully and reviewed by eye, and that is not the same as running them.
2. **The typecheck is UNRELIABLE and is NOT counted as a gate.** It completed several times and reported real errors; it later failed to complete on *every* attempt, including a two-file project, and a killed run exits 0 with empty output. The honest summary: **the type layer is not verified**. What follows is the record of what it found while it worked. Measured: a *one-file* project importing `react-native` took **182s** and was killed by the ~180s process cap, exiting 0 with empty output. That was only discovered because a deliberately broken file was *not* reported. On later attempts the same configuration completed and reported real errors.

Current state: the full check (42 files, all screens, real RN/Expo types) reports 0 diagnostics, confirmed on two consecutive runs that both printed their own summary line. Three genuine errors were found when the compiler last completed, and all three are fixed:

- `app/pick/[id].tsx` — a dangling `export type { EvidenceRow }` after the interface was removed
- `app/responsible-play.tsx` — `Body tone="caution"` was not in the union
- `app/settings.tsx` — `Body tone="verify"` was not in the union

Both tones were subsequently added, because a confirmation and a degraded condition are legitimate semantic states and the alternative (passing a colour) would have forked the theme. The unreliability is the caveat to keep: `scripts/typecheck.js` prints its own summary line precisely so a killed run is *visibly* a killed run.

3. **The three type errors that WERE caught** (before the compiler degraded) were real and are fixed: an `ErrorCode` comparison against a v2 constant that no longer exists, a `getStorefront({})` call with a spurious argument, and a `selectable` prop missing from the `Body` primitive. There may be more of the same class that were never surfaced.
4. **The server patches are unexecuted.** They import real modules and follow the repo's established patterns, but nothing ran them.
5. **No integration test exists between the client decoders and live server responses.** The decoders were written from the route sources; they have never seen real JSON.

Recommendation: the first action on a Mac is `npx expo start --ios`, then `tsc --noEmit`, then run the four server-patch routes against a local Postgres. Everything above is designed to make that a short session rather than a discovery session.

3. Defects found and fixed in this pass

4. Upstream findings — things in Beexly/Sports worth the owner’s attention

These were found while building and are **not** in the app’s gift to fix.

F1 — The FIELD palette collapses the four-band confidence ladder to two colours `DESIGN.md` specifies 80–100 plasma / 65–79 cyan / 50–64 UV / <50 silver. The FIELD revision retired cyan and UV to fog, so in the shipping tokens `--conf-strong`, `--conf-solid` and `--conf-lean` all resolve to `#C4BFB6`. Four bands, two effective colours. The app compensates with fill weight and an always-present numeral (which the contract requires anyway) rather than inventing hues, and the collapse is encoded in a test so it cannot go unnoticed. **Owner decision needed:** either accept a two-band ladder, or reintroduce two tints.

F2 — `DESIGN.md` and `design-system/` are still stale mirrors Their YAML/CSS still document the pre-FIELD palette (`plasma #FF2DD6`, `orbital cyan #00E5FF`, `ultraviolet #7A5CFF`) and, in the YAML’s case, the retired three-family type system. The port used `design-tokens.css`, which declares itself the sole authority. `BRAND_AND_DESIGN_SYSTEM.md` already flagged this as gap 1; it is still true, and now an agent has built from the *other* file, which is exactly the failure the flag predicted.

F3 — The X bot’s voice spec contradicts a finding it predates

`docs/product/twitter-bot-voice-spec.md` renders confidence as a percent: “*Published BOS –3.5 at 73% confidence*”. That predates the 2026-09-13 measurement that the 80+ band claims 0.8663 and realizes 0.5191 ($z = -10.7$). A percent on the widest-reach surface asserts a win probability the number demonstrably is not. `apps/web/lib/twitter-bot/templates/pick-publication.ts` implements the spec as written, so the template is currently primed to publish the error.

F4 — The same spec permits //⚖️ the design contract forbids emoji

Unresolved. `DESIGN.md` mandates W/L/P/V monograms and “emoji $\approx$ zero”. Both cannot be right. The recommendation is in the plan: allow the settlement glyph on the X lead post only, as a documented exception, and nowhere else.

F5 — There is no native auth path `auth()` reads a NextAuth cookie. A native client cannot present one. There is no bearer-token route. The patch in `server-patches/` fills this with a code-exchange flow and a signed session token; it is additive and touches no existing route.

F6 — `/api/push/subscribe` is Web Push only It validates a browser

`PushSubscription.toJSON()` shape (`endpoint` + `p256dh` / `auth`). A native app has none of those. Loosening the validator to accept both shapes would be how a validator stops validating, so the patch adds a separate route and a separate table.

F7 — React Native 0.86’s bundled TypeScript definitions are incomplete

`Libraries/Lists/FlatList.d.ts` extends `VirtualizedListProps`, but `ListHeaderComponent` and `ListFooterComponent` appear **nowhere** in the shipped types. Any strict-mode app using them fails to compile. The app sidesteps it by using `ScrollView` (justified independently — a slate is five rows), but the repo’s future React Native work should know.

F8 — expo-iap v3 renamed its error codes `E_USER_CANCELLED` became `ErrorCode.UserCancelled`. The v2 name is in most tutorials and in whatever model wrote the first draft. The failure mode is silent: a comparison against a non-existent string compiles (before the typechecker catches it) and makes every user cancellation render an error.

5. Doctrine compliance audit

Doctrine	Status	Evidence
Rule 8 — never frame the engine as AI	Pass	<code>lib/voice.ts</code> + linter; 0 violations over 41 files
Rule 3 — no frontend-only paywalls	Pass	<code>lib/entitlements.ts</code> header states it; a test asserts the module imports no components and holds no state
Rule 5 — no stale data	Pass	<code>FreshnessStamp</code> is required on every data surface; the cache returns an age and the UI prints “Offline copy — not refreshed”
Rule 7 — no <code>any</code>	Pass	Linter rule <code>types/no-any</code> , 0 hits
Rule 6 — tests	Partial	Logic tests exist and pass; there are no component tests (no RN test renderer on this host)
Confidence is never a percent	Pass	Linter rule + <code>confidenceScore()</code> returns <code>72/100</code>
Never render an adverse row	Pass	<code>applyBoardSafety</code> runs in one place; the comparator is imported, not restated
Settlement never colour-alone	Pass	Monogram + spelled-out a11y label on every badge
No casino green	Pass	<code>--verify</code> mint only; no green literal anywhere
44pt touch targets	Pass	<code>MIN_TOUCH_TARGET</code> in the primitive; linter cannot check this — verified by reading
Reduced motion	Pass	OS setting read once in <code>AppProviders</code> and OR-ed

Every `DateTimePicker` system-freshness

Pass

Required to pickCard, the design contract's never tokens, not the stale doc, override in components

5b. THE MOST EXPENSIVE GAP, found in research round 2

A purchase can succeed with the server never hearing about it, and the current code makes it unrecoverable.

`app/paywall.tsx` calls `finishTransaction` inside `onPurchaseSuccess`, immediately after the StoreKit success callback. `finishTransaction` tells StoreKit the app has handled the transaction, so StoreKit will **never re-deliver it**. If the subsequent server call fails, or the app is backgrounded, or the network drops, or the process is killed, then:

- the customer has been charged,
- the app has told StoreKit “handled”,
- and the server has no record, so the customer sees the free tier.

This is the single most expensive bug class in subscription software, and nothing in the current build closes the window. It is the highest-priority item in the app.

The fix, in shape (see `research/round-02-repositories.md` §1 for the reasoning):

1. **Do not finish the transaction until the SERVER acknowledges.** StoreKit re-delivers an unfinished transaction on next launch, which makes client-side retry logic unnecessary — the platform already has a durable queue and it is a mistake to build a second one.
2. **Set `appAccountToken`** to the app’s user id. `expo-iap`’s `PurchaseIOS` exposes it, and it is what makes a purchase made while signed out reconcilable to an account later.
3. **Reconcile on foreground.** `getAvailablePurchases()` against what the server knows, POST the difference. Idempotent, so it can run on every launch.

Recorded here rather than only in the cycle-2 queue because a reviewer reading this audit should not have to reach the end to find the thing most likely to cost real money.

6. Remaining gaps, in priority order

0. **Purchase reconciliation (see §5b).** Above everything else on this list.
1. **Build and run on a Mac.** Nothing else is meaningful until this happens.
2. **Typecheck the UI layer** (`tsc --noEmit`) on a host with a working compiler. Expect a handful of the class described in §4 F7/F8.
3. **Component tests.** `@testing-library/react-native` would cover the states that matter most (gate, stale, empty, redacted) — the ones a screenshot cannot prove.
4. **Wire `auth/exchange` into the web sign-in callback** (one line, documented in the patch).
5. **Apply the Prisma migration** for `MobileDevice` and `MobileAuthCode`.
6. **Live Activity.** `expo-widgets` is installed and the plugin stub exists (`plugins/withLiveActivity.js`); the widget target itself is not written.
7. **Audio briefings** (Phase 6 item 17). Scoped in the repo, not started here.
8. **Android.** Out of scope by the owner’s brief (iOS first), and the design language is dark-native, so it would port cleanly rather than need a redesign.
9. **The X bot’s transport.** Specified, templated, compliance-gated, operator-reviewable — and unable to post. See `docs/product/x-community-strategy.md`.